QoE Prediction from Encrypted Traffic with MARINA on P4/Tofino

Presenter: Elmer Dema (22211551)
Date:March 18th,2026
Supervisor: Prof. Andreas Kassler

Glossary

1- Motivation & Problem
2- pForest
3- Dataset
4- Quality of Experience
5- Implementation
6- Benchmarking

01

Motivation & Problem

Introduction

Video Boom: Video streaming dominates global IP traffic.

Importance of QoE: Quality of Experience (QoE) reflects true user satisfaction (e.g., stalling, buffering), which is vital for network operators.

The Challenge: Traffic is encrypted (HTTPS/QUIC), blinding traditional deep packet inspection.
State-of-the-art ML models (like **ViCrypt**) rely on end-host GPUs, which are too slow and unscalable for direct network data-plane implementation at terabit speeds.

Introduction

Traditional network monitoring relies on out-of-band analysis (NetFlow, sFlow), which introduces **high latency** and **sampling inaccuracies**.

The Solution: Moving Machine Learning (ML) directly into the data plane. By performing inference on the switch, we can achieve line-rate classification without CPU bottlenecks

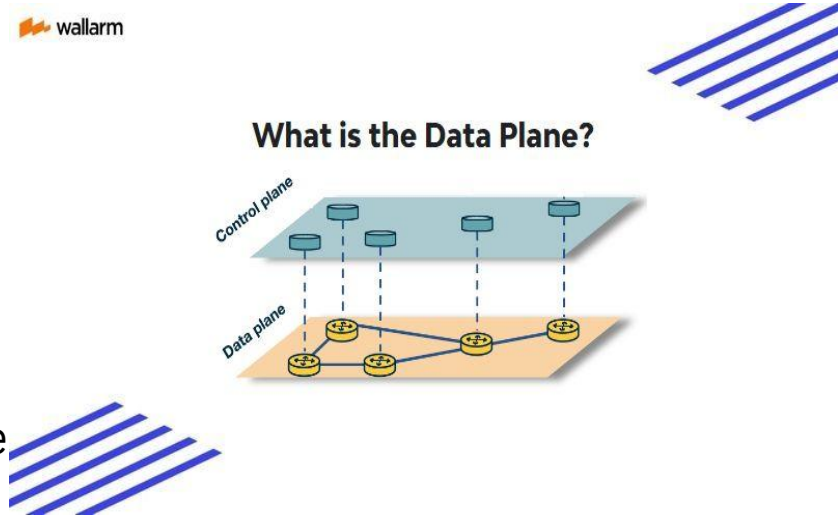


Figure 1: Data Plane

Our Approach

Decoupled Architecture: Reporter Switch: Performs initial video flow classification and extracts primitive features at line rate.

Translator Switch: Receives telemetry and runs a lightweight ML model (Random Forest) directly on the switch hardware to predict QoE.

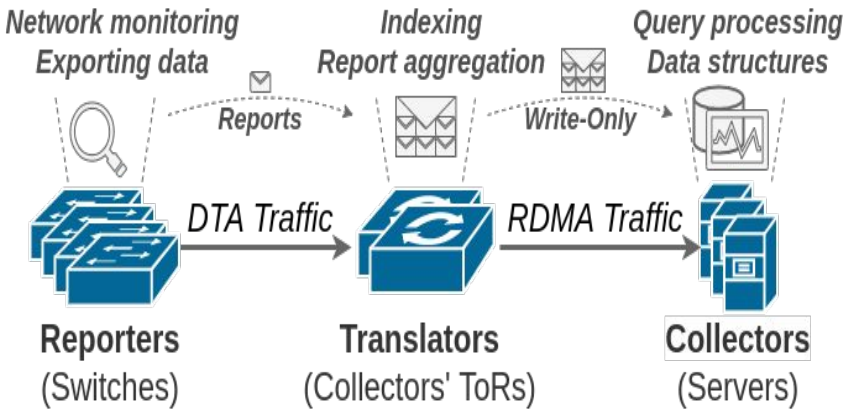


Figure 2 : Architecture example of Reporter/Translator

The Challenge

Tofino Constraints: The Intel Tofino uses a pipeline architecture . It has no floating-point units, no loops, and strictly limited memory (SRAM)

Standard ML models (like scikit-learn) assume infinite memory and float precision. We must map these to **Match-Action Tables and stateful registers**

Figure 3

Figure 4

pForest: In-Network Inference with Random Forests

C. Busse-Grawitz et al., "pForest: In-Network Inference with Random Forests," ACM SIGCOMM, 2022.

Figure 5

02

pForest

What is pForest?

A framework designed for programmable network data planes that executes **Random Forest machine learning models** directly on the switch to perform real-time, **line-rate traffic classification**.

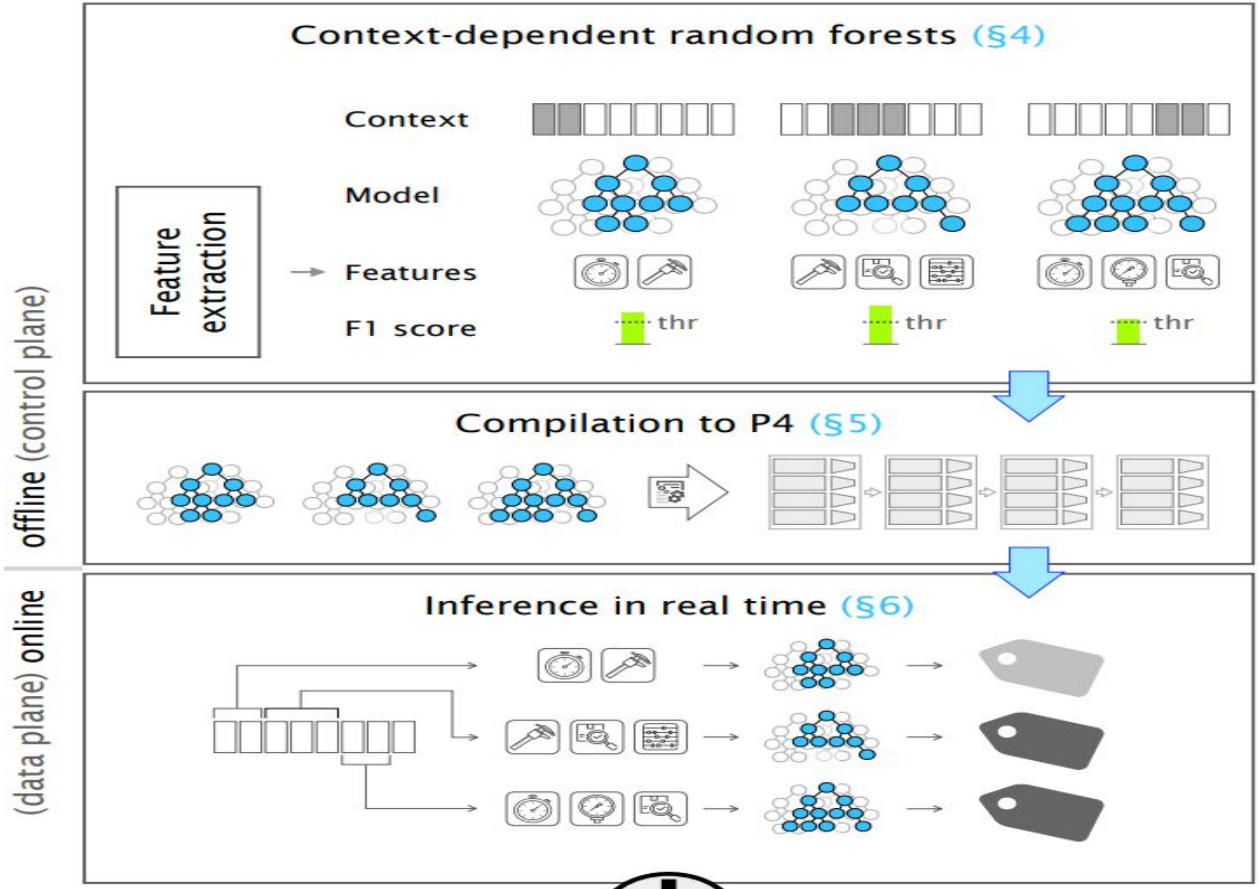
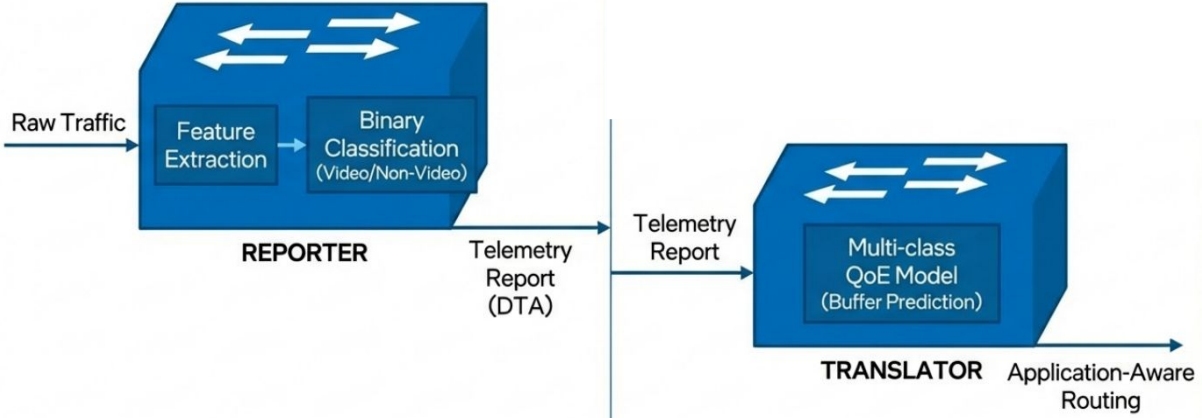

Context-dependent random forests (§4)
Context
Model
Features
F1 score
Feature extraction
thr
thr
thr
Compilation to P4 (§5)
Inference in real time (§6)
offline (control plane)
(data plane) online

Figure 6

Feature Extraction

- Features are computed and stored in stateful registers (using the Marina Reporter logic) as packets arrive.
- When we generate a telemetry packet the pipeline reads the pre-computed values (e.g., packet_length_mean, flow_IAT_max) from the registers rather than computing them on the fly.



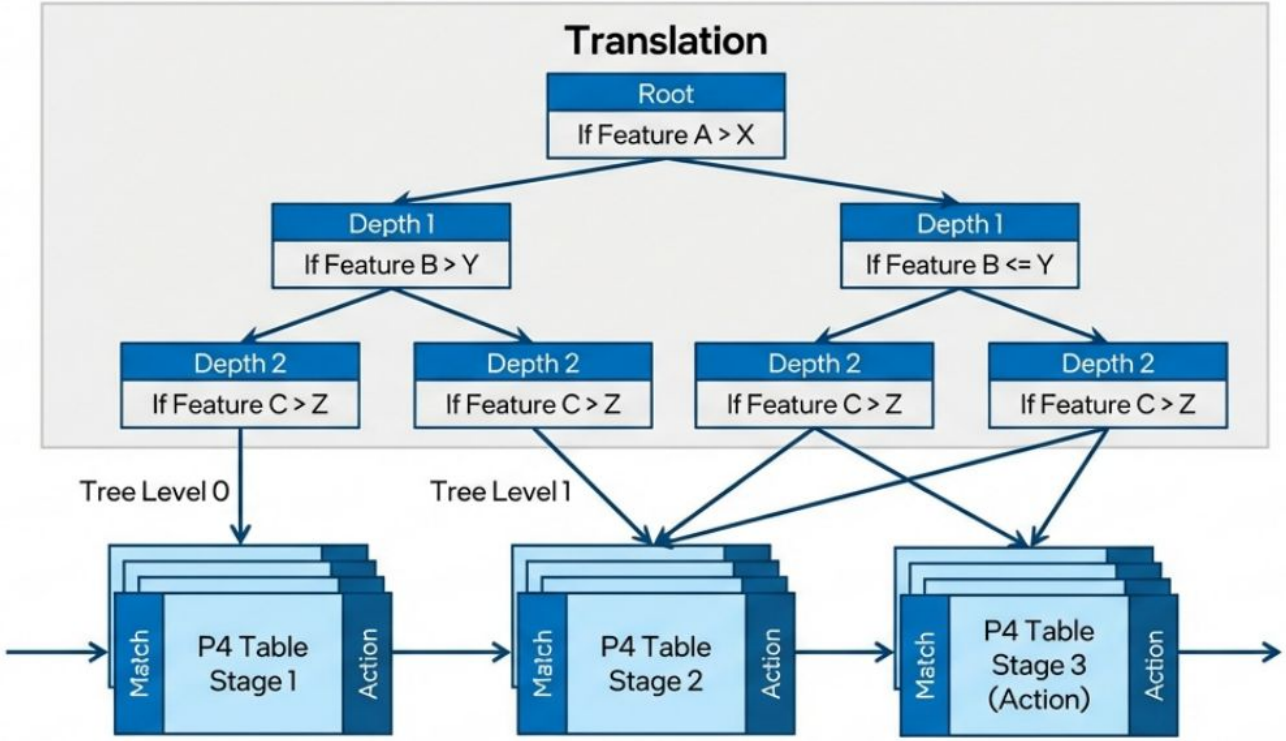
Compiling Decision Trees to Match-Action Tables

P4 does not support if/else trees natively. We map each level of a decision tree to a specific Match-Action Table.

Tofino Pipeline Architecture (TNA) limitations:
The Tofino switch processes packets through a fixed sequence of stages (typically 12 on Tofino 1)

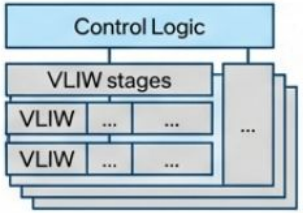
We use registers to store flow features and the hardware limits us to only 4 register actions in parallel per stage.

Compiling Decision Trees to Match-Action Tables



Technical Notes:

- **Quantization:** Features compressed to fit Packet Header Vector (PHV).
- **Parallelism:** Multiple trees evaluated simultaneously in VLIW stages.
- **Optimization:** Depth restricted to 3-5 stages per tree.



03

Dataset

Dataset

The data used to train and validate this system comes from the **Karagkioules et al. dataset**, which captures YouTube native app traffic under various network conditions.

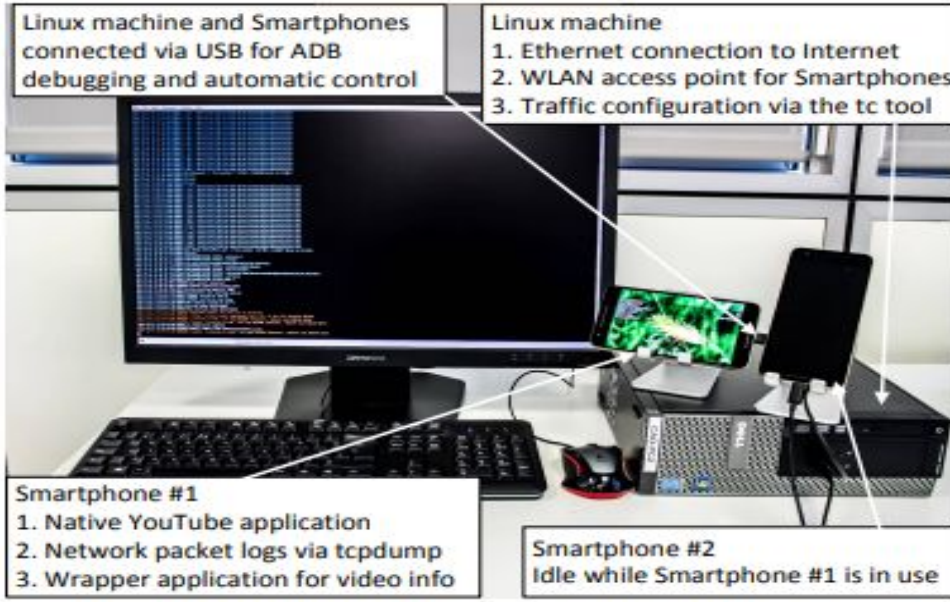


Figure 7

Dataset

Scenarios 1, 2, and 3 impose constraints by forcing the video quality to fixed resolutions (**480p** or **720p**). In contrast, Scenario 6 uses "Auto" quality, This reflects how real users interact with platforms like YouTube.

Scenario	Quality	Rate limit
s1	480p	No
s2	720p	No
s3	$720p \xrightarrow{t_0} 480p$ $t_0 \in [120, 240]\ s$	No
s4	Auto	$\text{No} \xrightarrow{t_0} 500\ \text{kbit/s} \xrightarrow{t_1} \text{No}$ $t_0 \in [120, 240]\text{s},\ t_1 = t_0 + 150$
s5	Auto	1024 kbit/s
s6	Auto	(see Figure 3)
s7	Auto	$\text{No} \xrightarrow{120s} 3000\ \text{kbit/s} \underset{t_1}{\overset{t_0}{\rightleftarrows}} 100\ \text{kbit/s}$ $t_0 = 160 + 85n, n \in \mathbb{N}$ $t_1 = 205 + 85n, n \in \mathbb{N}$
s8	Auto	$\text{No} \underset{t_1}{\overset{t_0}{\rightleftarrows}} 100\ \text{kbit/s}$ $t_0 = \{120, 300\}\text{s},\ t_1 = \{180, 380\}\text{s}$

Figure 8

04

Quality Of Experience (QoE)

What is Quality of Experience (QoE)

QoE measures performance from the user's subjective perspective, it focuses on visible impacts such as stalling or resolution drops rather than just packet statistics.

Essentially, Quality of Service (**QoS**) measures network speed, while **QoE** measures user satisfaction.

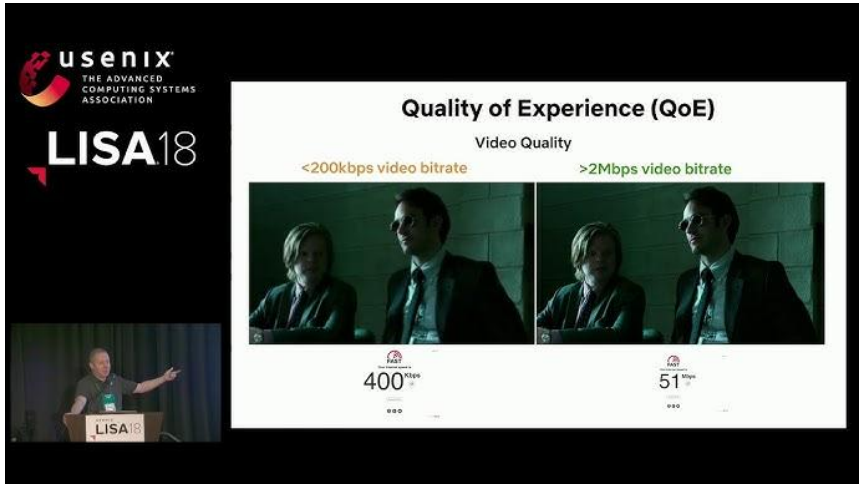


Figure 9

Our Goal

Feature Name	Mathematical Derivation
Packet Count	Count of events in window
Traffic Volume (PS)	$\sum \mathrm{PS}_i$
Traffic Moments	$\mathrm{PS}^2, \mathrm{PS}^3$
IAT Total	$\Delta t = t_i - t_{i-1}$
IAT Moments	$\mathrm{IAT}^2, \mathrm{IAT}^3$
Jitter	$\lvert \mathrm{IAT}_i - \mathrm{IAT}_{i-1} \rvert$

These are some of the features which are extracted. In our case we want to predict buffer (in ms). **But how can a continuous variable be useful in our case?**

Target Classes

We attempted a simple binary classification based on whether the buffer was filling or depleting. However, this lacked context. So that is when we decided to implement other QoE States.

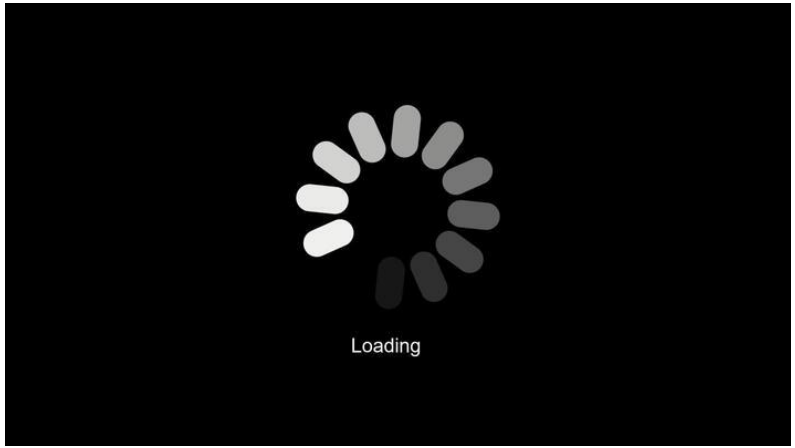


Figure 10

Target Classes

- A risk-based classification scheme using a "Lookahead" mechanism was implemented.
- A Critical state was defined when the buffer is empty and draining rapidly, distinguishing it from Safe or Steady states and prioritizing detection of imminent video stalls.

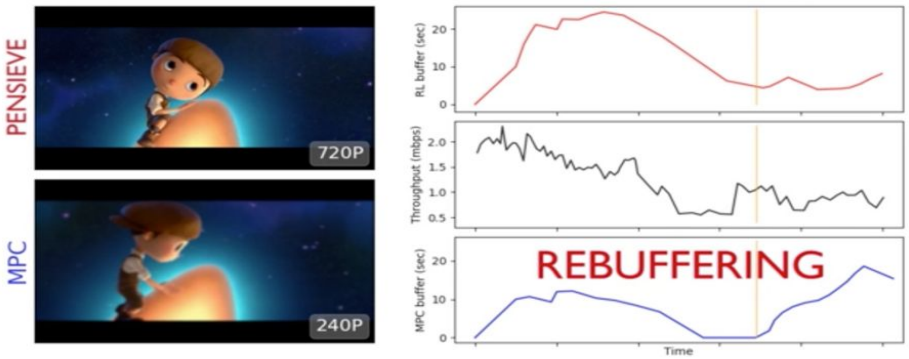


Figure 11

Model Training

To establish a baseline for classification performance, a Random Forest Classifier was trained using 150 estimators and a maximum depth of 12.

The input feature vector for this experiment was reduced to four core metrics: **packet_count, ps_sum, iat_sum, and jitter.**

Feature	Importance Score
Traffic Volume (ps_sum)	0.354
Jitter (jitter)	0.302
IAT Sum (iat_sum)	0.186
Packet Count	0.158

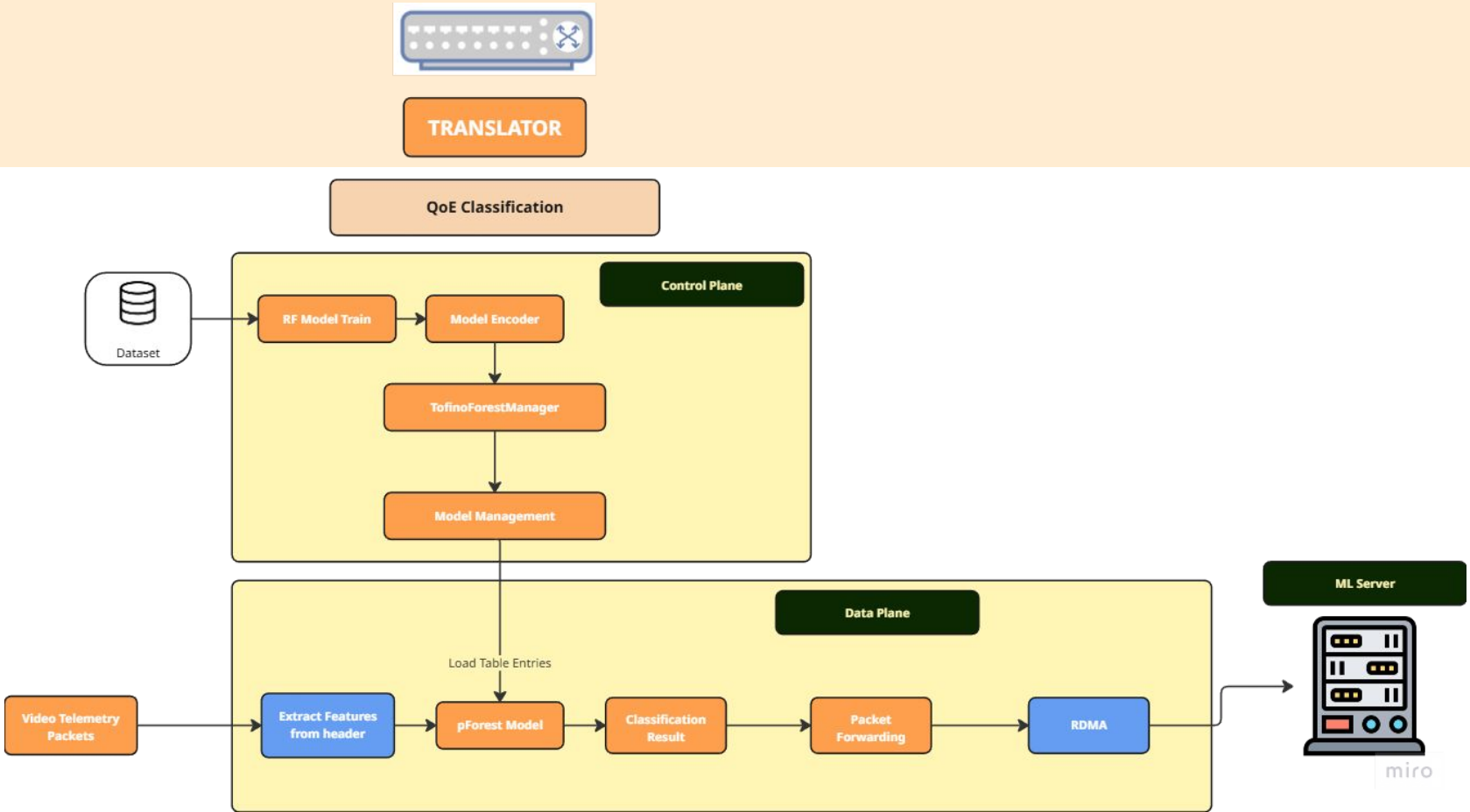
Model Training

After testing locally, we got a model with ~80% accuracy. As shown in the classification report, the model struggled significantly with the “Critical” class due to the scarcity of samples (only 19 support instances in the test set).

The dataset was then exported to a **.npy** file which will be used by **pForest**

Class	Precision	Recall	F1-Score
Critical	0.08	0.95	0.00
Safe_Drain	0.78	0.02	0.03
Steady	0.69	0.69	0.69

New Translator Architecture



05

Switch Implementation

What is TREX

TRex is a software traffic generator developed by Cisco, designed to simulate network traffic at high speeds. In this thesis, TRex served as the core testing engine within the hardware-in-the-loop testbed to replay realistic network traces (PCAP files) and synthetic telemetry packets into the Intel Tofino switch.

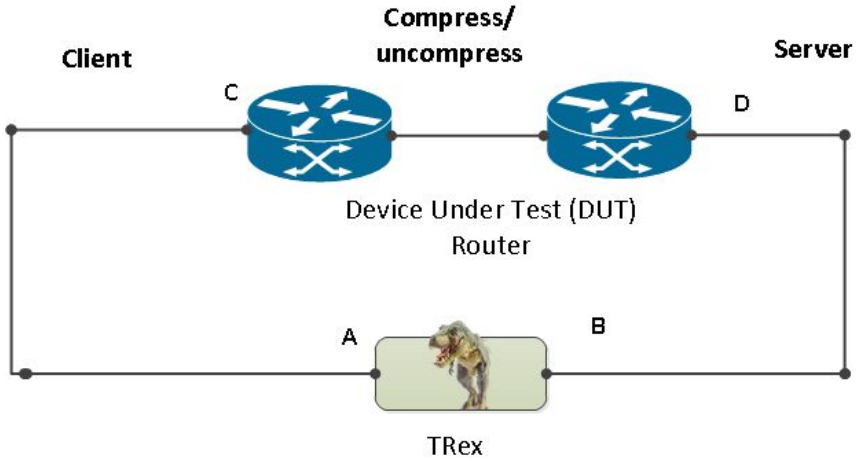


Figure 12

Telemetry Generation (Reporter)

- **TRex** : Replayed raw TCPdump logs (Scenario 6) into the Tofino switch.
- **Tofino Switch** (Reporter P4 Program): Processed incoming raw traffic and aggregated stateful network features. Generated and emitted a dedicated DTA telemetry packet every 100ms.
- **Capture** & **Output**: Routed the DTA packets back to TRex. Captured a 28.5 MB PCAP file (trex_tofino_reply.pcap) containing 35 purely synthetic telemetry reports to serve as ground-truth input.

PCAP file

Index	Source	Target	Protocol	Size	Info
1	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
2	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
3	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
4	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
5	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
6	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
7	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
8	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
9	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
10	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
11	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
12	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
13	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34
14	192.168.123.1	192.168.100.1	udp	76	1025 → 4500 Len=34

Real-Time ML Inference (Translator

Tofino Switch (Translator P4 Program): Ran Random Forest classification on incoming DTA packets at line rate. Appended the final QoE classification result to the packet header and executed state-dependent routing based on the ML prediction.

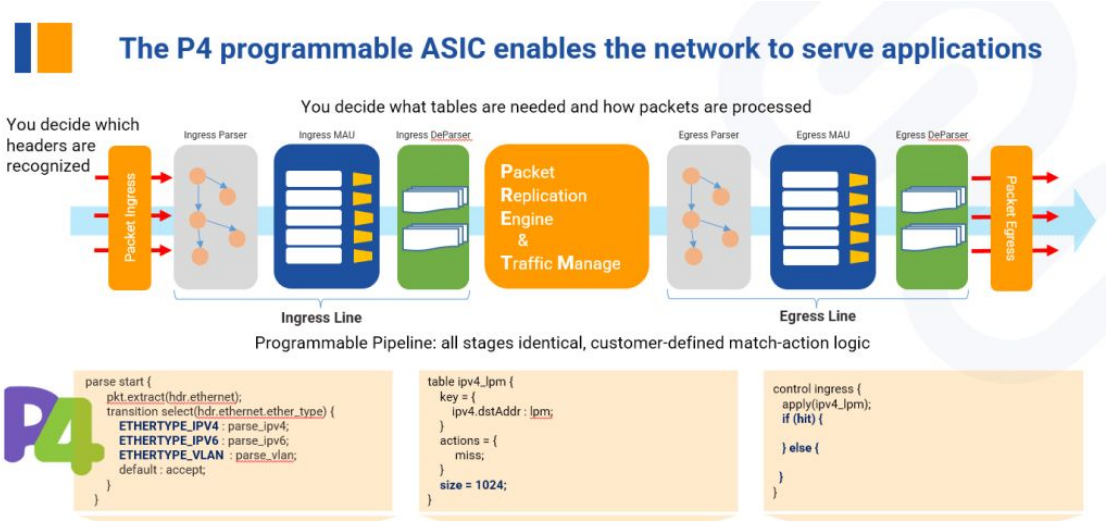


Figure 13

DTA Metadata

Telemetry Field	Decoded Hardware Value
L2 MAC Address	d0:77:ce:2b:20:54 -> 10:70:fd:30:80:d1
L3 IP Address	192.168.123.1 -> 192.168.100.2
L4 Protocol	UDP 49374 -> 40040 (DTA)
— DTA Header Metadata —	
Opcode	0x05
Redundancy Level	1
Telemetry Key (Flow ID)	2980564395
Monitored Flow	192.168.123.1:1025 -> 192.168.100.1:4500 (UDP)
Packet Count	198
Last Packet Timestamp	850,630,301
Sum of IAT	99,952,039
Sum of IAT^2 / IAT^3	0 / 0
Sum of Packet Size	272,844
Sum of Packet $Size^2$ / $Size^3$	0 / 0
Jitter	2,001,694
Classification Result	**0** (NON-VIDEO)
Used Features Bitmap	0x0000000d

After classification

The switch uses the predicted QoE state (final_class) to decide where the packet goes next:

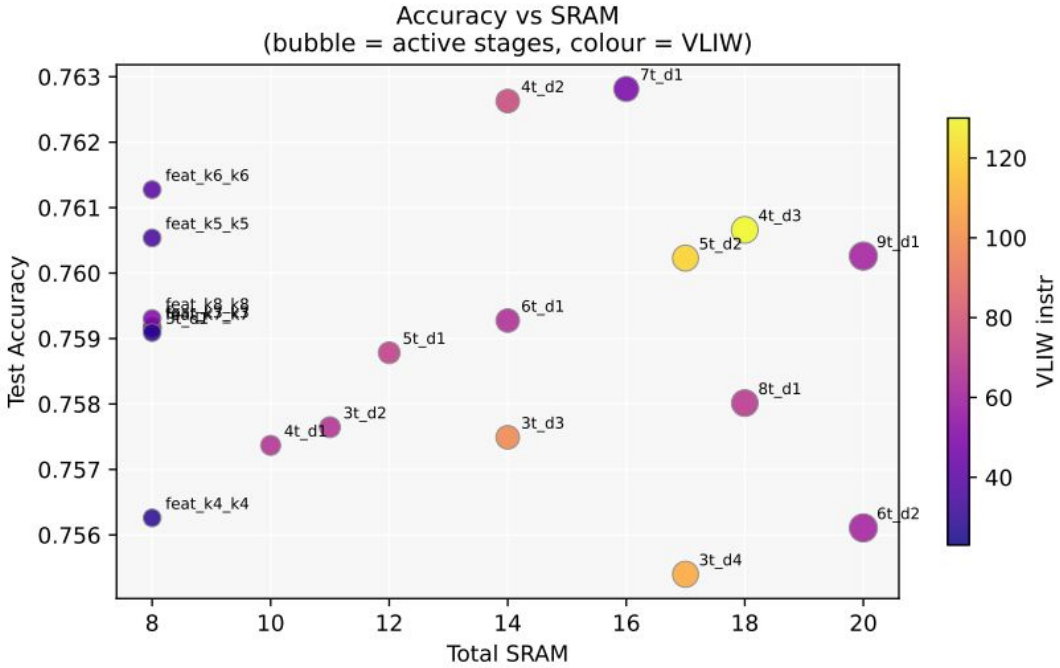
If classified as **Benign/Steady (Class 0)**: The switch executes normal routing (e.g., forwards the traffic along its normal path to a specific port).

If classified as **Critical/Malicious (Class 1)**: It can either redirect the packet to a specific port for Deep Packet Inspection (DPI) or immediately drop/quarantine the flow to prevent network congestion or stalling.

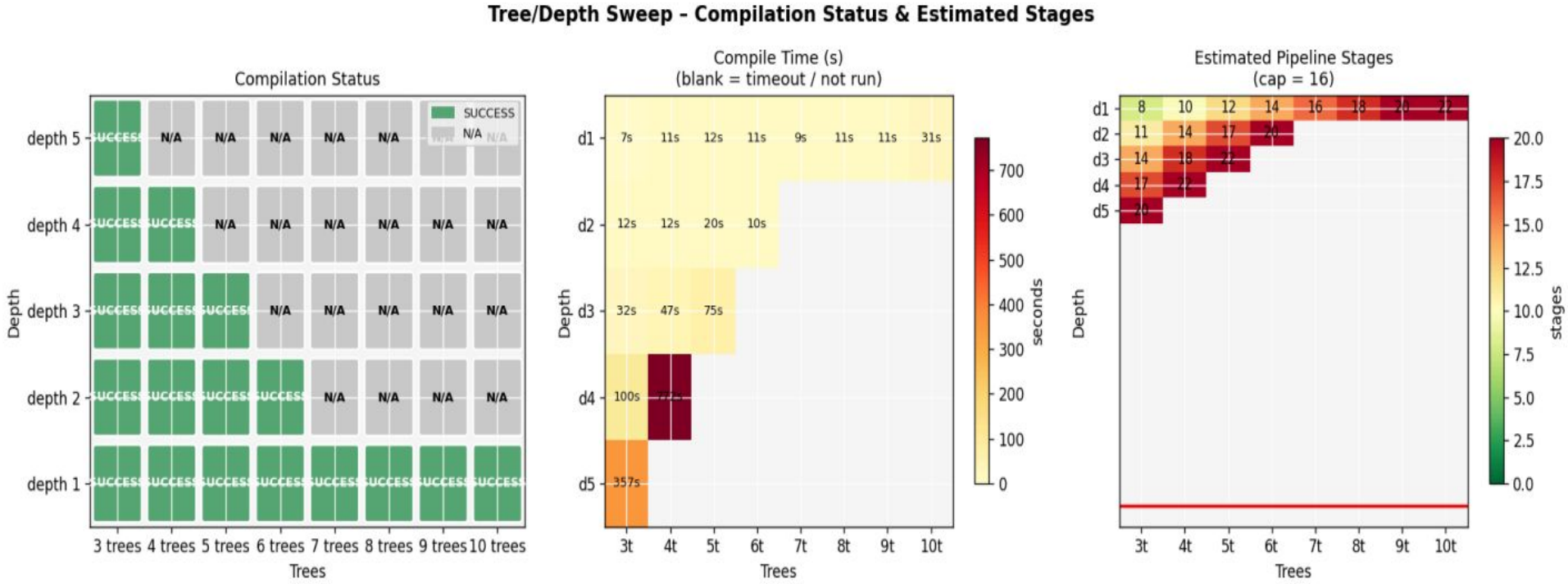
06

Benchmarking

Benchmarking



Benchmarking



Conclusions

Key Achievements

In-Network ML: Successfully bridged high-level machine learning with low-level P4 data plane programming.

Line-Rate Inference: Proved QoE can be predicted from encrypted traffic at terabit speeds without relying on external GPUs.

Better luck next time. (Cons)

Hardware Constraints Pipeline Bottlenecks: The Tofino 2 has a hard physical limit of 20 pipeline stages.
Dataset: Creating our own dataset would have improved our accuracy

Future Work Packet Recirculation: Bypassing the 20-stage limit by recirculating packets through the switch pipeline to allow for deeper, more accurate decision trees

Figure 14

Repository

elmerdema final commit 5186923 · 2 days ago 42 Commits

benchmarks	Add benchmark report, alongside updating the gitignore.	3 days ago
code	refactor: remove unnecessary comments and clean up code s...	2 days ago
p4	added readme, added trex scripts, updated pdf	2 days ago
scripts	final commit?	2 days ago
templates	Add deployment scripts and traffic generator for Tofino Trans...	3 days ago
typst/0.1.4	final commit	2 days ago
.gitignore	Add benchmark report, alongside updating the gitignore.	3 days ago
.python-version	Add initial project structure with README, Python version, m...	5 months ago
README.md	added readme, added trex scripts, updated pdf	2 days ago
pyproject.toml	added eda to ipynb and updated lock file	4 months ago
uv.lock	added eda to ipynb and updated lock file	4 months ago

https://github.com/elmerdema/thesis

Bibliography/ Images

- C. Busse-Grawitz et al., "pForest: In-Network Inference with Random Forests," ACM SIGCOMM, 2022.
- L. Froschauer, J. Langlet, and A. Kassler, "Direct Feature Access — Scaling Network Traffic Feature Collection to Terabit Speed," arXiv:2505.17573 [cs.NI], 2025.
- F. Loh et al., "Demo: A Wrapper for Automated Measurements with YouTube's Native App," IEEE/IFIP NOMS, 2018.
- S. Wassermann and et al., "ViCrypt to the Rescue: Real-Time, Machine-LearningDriven Video-QoE Monitoring for Encrypted Streaming Traffic."
- O. Izima, R. de Fréin, and M. Davis, "Video Quality Prediction Under Time-Varying Loads." Nicosia, Cyprus, 2018.
- M. T. Vega, C. Perra, and A. Liotta, "Resilience of Video Streaming Services to Network Impairments," vol. 64. pp. 220–234, 2018.
- Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," vol. 13. 2004.
- M. Seufert and et al., "A Wrapper for Automatic Measurements with YouTube's Native Android App." June 2018.
- T. Karagkioules et al., "A Public Dataset for YouTube's Mobile Streaming Client."
- roxennnn, "pForest Source Code." [Online]. Available: https://github.com/roxennnn/pforest/tree/master/src

Bibliography/ Images

Figure 1:wallarm.com/what/what-is-the-data-plane
Figure 2:Image:github.com/jonlanglet/DTA
Figure 3:https://scikit-learn.org
Figure 4: https://github.com/roxennnn/pforest
Figure 5: https://www.intel.com/content/www/us/en/products/network-io/programmable-ethernet-swit
Figure 6: https://github.com/roxennnn/pforest
Figure 7: https://www.nature.com/articles/s41597-022-01418-y
Figure 8:irp-cdn.multiscreensite.com/d3de1972/files/uploaded/Conference_51_Michael_Seufert.pdf
Figure 9:youtube.com/watch?v=nHG8d14FWxY
Figure 10: vecteezy.com/video/44175645-buffering-loading-animation-black-screen
Figure 11: youtube.com/watch?v=nHG8d14FWxY
Figure 12: https://trex-tgn.cisco.com/trex/doc/trex_astf.html
Figure 13: https://cloudswit.ch/blogs/what-you-should-know-about-p4-programming-language-p4-switc
Figure 14:https://tenor.com/view/shrug-idk-elmo-puppet-better-luck-next-time-gif-12629800

Thanks for your attention

Questions?